Osciloscopio Básico Monocanal

Basic Single-Channel Oscilloscope

[Facundo Nicolas Nestasio][1] · [Sebastian Canteros][2]

[1] UTN, FRA, Avellaneda, Buenos Aires, Argentina · ORCID: N/A

[2] UTN, FRA, Avellaneda, Buenos Aires, Argentina · ORCID: N/A

Resumen

El proyecto que detalla este informe consiste en el diseño e implementación de un osciloscopio de memoria digital monocanal. En la primera fase se desarrolla el instrumento, basado en un microcontrolador ESP32 ejecutando el sistema operativo en tiempo real FreeRTOS. Se propone una arquitectura de adquisición continua mediante Acceso Directo a Memoria (DMA) y un sistema de sincronismo implementado por software para permitir la visualización de la señal con función de predisparo. La interfaz de usuario consta de un encoder rotativo y transmisión de datos para visualización externa. La segunda fase se centra en establecer una comunicación UART a nivel de kernel con el procesador BCM2711 (ARM Cortex-A72, ARMv8, 64 bits) de una Raspberry Pi 4, que ejecuta Raspberry Pi OS de 64 bits basado en Linux kernel v6.12.

Palabras clave: Osciloscopio; FreeRTOS; ESP32; DSO; DMA; Trigger; Raspberry Pi 4; Linux.

Abstract

The project detailed in this report consists of the design and implementation of a single-channel digital storage oscilloscope. In the first phase, the instrument is developed based on an ESP32 microcontroller running the FreeRTOS real-time operating system. A continuous acquisition architecture utilizing Direct Memory Access (DMA) and a software-implemented trigger system is proposed to enable signal visualization with a pre-trigger function. The user interface consists of a rotary encoder and data transmission for external display. The second phase focuses on establishing a kernel-level UART communication with the BCM2711 processor (ARM Cortex-A72, ARMv8, 64-bit) of a Raspberry Pi 4, running the 64-bit Raspberry Pi OS based on Linux kernel v6.12.

Keywords: Oscilloscope; FreeRTOS; ESP32; DSO; DMA; Trigger; Raspberry Pi 4; Linux.

Índice

1. Introducción **3**

2. Desarrollo **3**

2.1. Objetivos . . . 3
2.2. Alcance . . . 3
2.3. Fundamentación . . . 3
2.4. Diagrama en bloques . . . 5
2.5. Diagrama de tareas . . . 5
2.6. Diagrama de Software . . . 5
2.7. Hardware . . . 5
2.8. Software . . . 10

3. Resultados **12**

3.1. Mediciones . . . 14

4. Conclusiones **15**

5. Referencias **15**

Anexo A: Diagramas del sistema **16**

Anexo B: Esquemático y PCB **19**

Anexo C: Manual de usuario y dispositivo **21**

Anexo D: Listado de comandos **25**

Anexo E: Muestra de Mediciones **26**

1. Introducción

Problemática

En el desarrollo y diagnóstico de sistemas electrónicos, es indispensable contar con instrumentos capaces de registrar y visualizar señales variables en el tiempo. Los osciloscopios de laboratorio tradicionales ofrecen altas prestaciones, pero su costo, tamaño y complejidad limitan su accesibilidad para entornos de desarrollo ágil o mediciones portátiles.

Solución

Frente a esta problemática, se propone el diseño de un osciloscopio digital compacto y de bajo costo utilizando el microcontrolador ESP32. Al trasladar funciones típicamente resueltas por hardware, como el sincronismo, al dominio del procesamiento por software, se reduce drásticamente la cantidad de componentes externos. Esto permite mantener una alta flexibilidad en la configuración y optimizar el presupuesto del instrumento, aprovechando el alto poder de cómputo y la accesibilidad del microcontrolador seleccionado.

2. Desarrollo

2.1. Objetivos

- Procesar y transmitir señales analógicas para su visualización a través de un osciloscopio digital de forma segura.
- Implementar un trigger por nivel configurable con selección de flanco.
- Implementar dos escalas verticales seleccionables.
- Implementar tres bases de tiempo seleccionables.
- Visualizar la señal en una PC a través de un software de visualización.
- Implementar la gestión de los comandos remotamente mediante un GPOS.

2.2. Alcance

Se pretende lograr un osciloscopio digital monocanal con los siguientes parámetros:

- **Escalas verticales (amplitud):** 1 V/div y 5 V/div.
- **Escalas horizontales (base de tiempo):** 1 ms/div, 2 ms/div y 5 ms/div.
- **Atenuación de entrada:** x1 y x10.

2.3. Fundamentación

Software

Para el desarrollo de este proyecto se optó por utilizar el sistema operativo en tiempo real FreeRTOS. Este entorno facilita la organización de los flujos de trabajo y las rutinas en tareas independientes con prioridades asignadas, permitiendo una ejecución concurrente eficiente mediante el uso de mecanismos tales como colas de mensajes y semáforos de eventos.

En lo que respecta a la etapa de digitalización, el muestreo de la señal analógica se realiza mediante el conversor analógico-digital (ADC) nativo del microcontrolador, el cual posee una resolución de 12 bits. Considerando que los conversores de los osciloscopios digitales comerciales operan típicamente con una resolución de 8 bits a 16 bits, se determina que la resolución del dispositivo es satisfactoria para la tarea.

Por otro lado, la interfaz y las acciones del usuario se gestionan mediante interrupciones externas por GPIO, las cuales envían los eventos de configuración hacia el sistema a través de la cola `q_scale`. Finalmente, para la representación gráfica de la señal en el dominio del tiempo, los datos procesados son despachados por el puerto UART mediante una cola de visualización, permitiendo su ploteo en tiempo real a través de un software hecho en Python utilizando la librería pyqtgraph.

En el entorno Linux, se desarrollará un módulo de kernel encargado de controlar el puerto UART, permitiendo la comunicación con el sistema embebido que gestiona el control del osciloscopio ysu visualización. El control se realizará mediante una interfaz de consola en la Raspberry Pi 4 mediante la cual se solicitaran comandos normalizados GET/SET.

Hardware

Dado que los pines del conversor analógico-digital del microcontrolador operan estrictamente en el rango de 0 V a 3,3 V y no admiten tensiones negativas, es necesario implementar una etapa previa de adaptación. Esta etapa (compuesta por atenuadores, amplificadores operacionales y divisores resistivos) cumple dos funciones críticas: escalar la amplitud de la señal de entrada para que coincida con la escala vertical seleccionada por el usuario, y sumar un nivel de tensión continua (offset) de 1,65 V. De este modo, una señal alterna pura oscilará de manera segura dentro de la ventana de lectura del microcontrolador, protegiendo el silicio contra sobretensiones y permitiendo la reconstrucción matemática posterior del semiciclo negativo.

El núcleo de la digitalización explota las capacidades del ADC interno de 12 bits de resolución operando de forma ininterrumpida. Al configurar el ADC para que sea disparado constantemente por un temporizador por hardware, el controlador de Acceso Directo a Memoria (DMA) transfiere las muestras hacia arreglos de memoria RAM de manera automática y cíclica. Al operar con este flujo continuo de datos, el sistema delega la decisión de captura a la tarea de sincronismo (Trigger); es esta lógica la que evalúa la memoria en tiempo real y, al detectar el evento de interés, emite la orden de congelar y guardar el búfer actual para su procesamiento.

Para la interfaz de control se optó por utilizar un encoder que envíe las entradas del usuario a través de GPIO y una pantalla OLED donde el usuario pueda visualizar la configuración, el impacto

de su entrada y los datos de interés. La señal se visualizará a través de una PC que recibirá los datos de la señal por UART.

La incorporación de la computadora monoplaca Raspberry Pi 4 Model B responde a la necesidad de dotar al instrumento de una capa de procesamiento robusta y conectividad de red remota. Gracias a su procesador ARM Cortex-A72 y la capacidad de ejecutar un sistema operativo Linux completo, la placa asume el rol de mediador entre la adquisición de bajo nivel y la visualización final.

2.4. Diagrama en bloques

En la Figura 12 se observa el esquema en bloques del osciloscopio, donde se detallan las entradas y salidas de los bloques y sus conexiones. Aquellos en los que no se especifican las entradas y salidas es porque se detallan en el diagrama de tareas.

2.5. Diagrama de tareas

En la Figura 13 del Anexo A se observa el diagrama de tareas de FreeRTOS, donde se detalla la funcionalidad de cada una de ellas, sus prioridades y su interacción con las demás mediante colas y eventos registrados en semáforos.

2.6. Diagrama de Software

El sistema se divide en cuatro capas. En el ESP32, la tarea vComandoUartTask queda a la espera de datos en el buffer del UART. Cuando recibe un comando completo, actualiza la configuración y sobrescribe los datos de config_queue. Esta tarea se comunica por UART con la Raspberry Pi mediante comandos GET/SET.

En la Raspberry Pi, el kernel expone el UART como un dispositivo de caracteres. El *char device* es un archivo en `/dev`, identificado por un número major/minor y registrado con `register_chrdev`. Es el punto de entrada para las operaciones open, read, write y release. El driver UART registra esas *file operations* y configura el puerto según el *device tree overlay*. Cuando llega un byte nuevo por TX, se dispara una interrupción (IRQ). Su *handler* guarda el dato en el buffer del driver.

En el espacio de usuario, la aplicación de consola abre el archivo de dispositivo y arma los comandos GET/SET. Por último, la aplicación gráfica, hecha en Python y ejecutada en la PC, se conecta a la consola por SSH usando Paramiko. Desde ahí permite controlar los parámetros del osciloscopio y visualizar la señal de forma remota.

En la Figura 14 del Anexo A podemos ver el diagrama del software.

2.7. Hardware

ESP32-S3

- Microcontrolador de *Espressif Systems* con arquitectura Xtensa LX7 de doble núcleo.

- Frecuencia de funcionamiento máxima de 240 MHz.
- Tensión de alimentación de 3,0 V a 3,6 V.
- Hasta 45 GPIO configurables.
- Incorpora convertidores analógico-digitales (ADC) de 12 bits, utilizados para la adquisición de las señales de entrada del osciloscopio.
- Interfaces de comunicación UART, SPI, I2C, I2S, CAN (TWAI) y USB OTG.

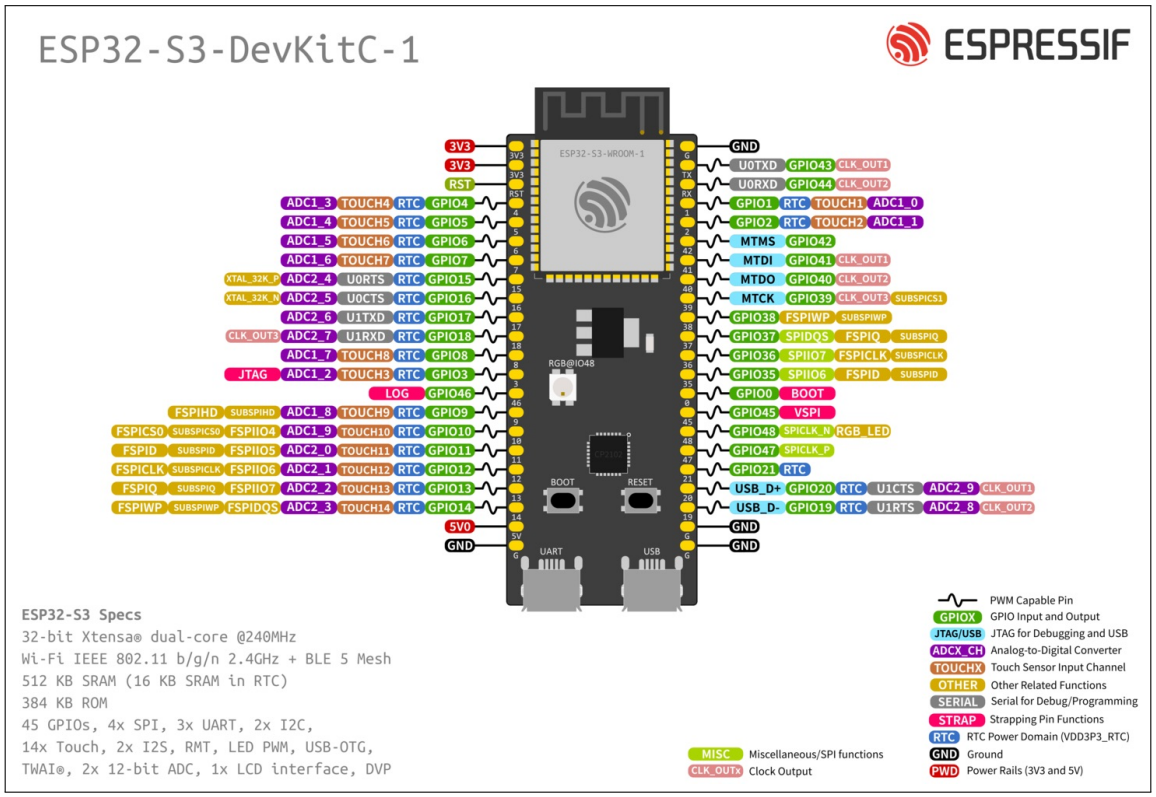


Figura 1: ESP32-S3 Pinout.

Etapa de atenuación

Para la etapa de atenuación X1 se podrán medir tensiones de hasta 3,3 V sin divisor resistivo. Para llegar a 33 V se utilizará un divisor resistivo, debido a que el ADC del ESP32-S3 admite hasta 3,3 V.

$$33\,\mathrm{V} \cdot \frac{R_2}{R_1 + R_2} = 3{,}3\,\mathrm{V} \tag{1}$$

$$R_2 = 10\,\mathrm{k\Omega} \tag{2}$$

$$R_1 = 90\,\mathrm{k\Omega} \tag{3}$$

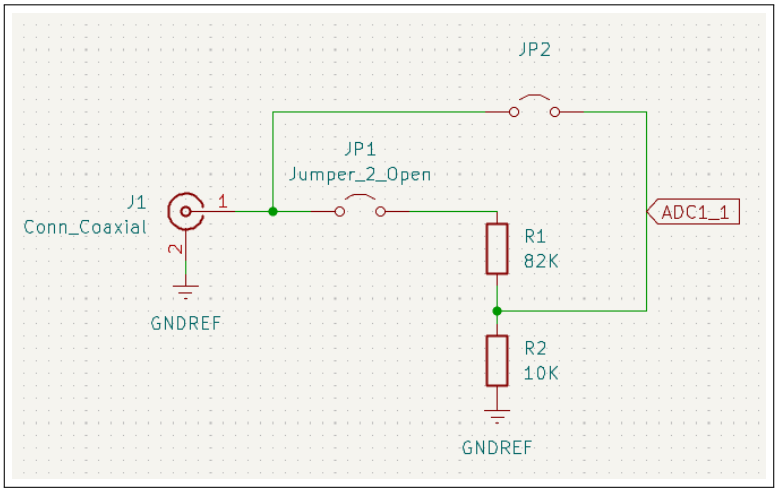


Figura 2: Etapa de atenuación.

Además, para poder soportar las señales negativas se debe implementar una señal de offset. Para eso se utilizará el pin de 3,3 V del micro y un operacional en la configuración de sumador no inversor. El operacional utilizado será el TL074 y se utilizará el siguiente circuito:

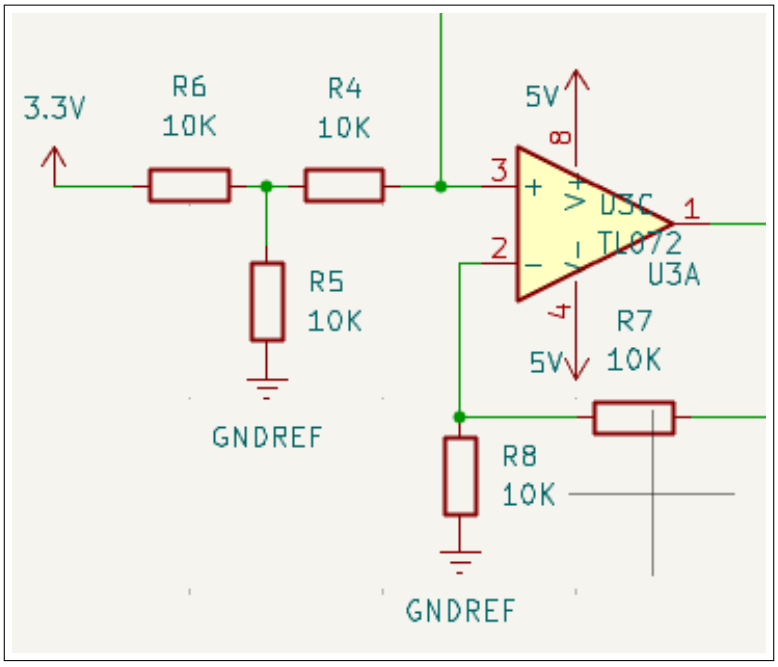


Figura 3: Etapa de offset (sumador no inversor).

Para el circuito de protección contra sobretensiones, utilizaremos dos diodos Schottky, uno conectado a una tensión de referencia de 3,3 V y otro conectado a masa. Esto hará que cuando la señal sea negativa se limite la tensión de entrada a 0,2 V y que cuando la señal exceda el rango del osciloscopio se la limite a 3,5 V.

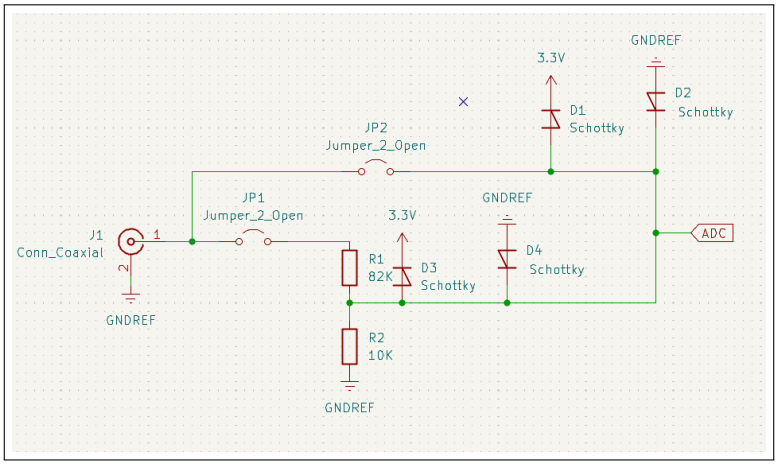


Figura 4: Etapa de protección.

Módulo relé de 2 canales

Para la conmutación entre los modos de entrada del osciloscopio (X1, X10 y AC) se utilizó un módulo de dos relés con optoacoplador, controlado desde el ESP32-S3:

- Voltaje de alimentación: 5 V.
- Relés: 2 × Songle SRD-05VDC-SL-C, con bobina de 5 V.
- Capacidad de los contactos: 10 A a 250 V AC / 30 V DC.
- Activación por nivel bajo en las entradas IN1 e IN2

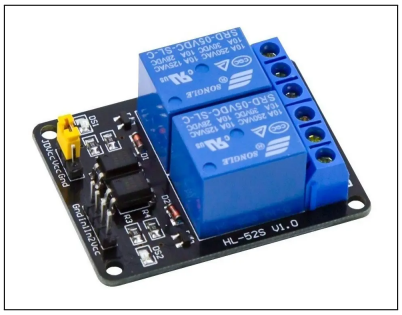


Figura 5: Modulo rele 2 canales.

Display OLED SSD1306

- Tensión de alimentación de 3,3 V.
- Resolución de 128×64 píxeles.
- Pantalla de 35×24 mm.

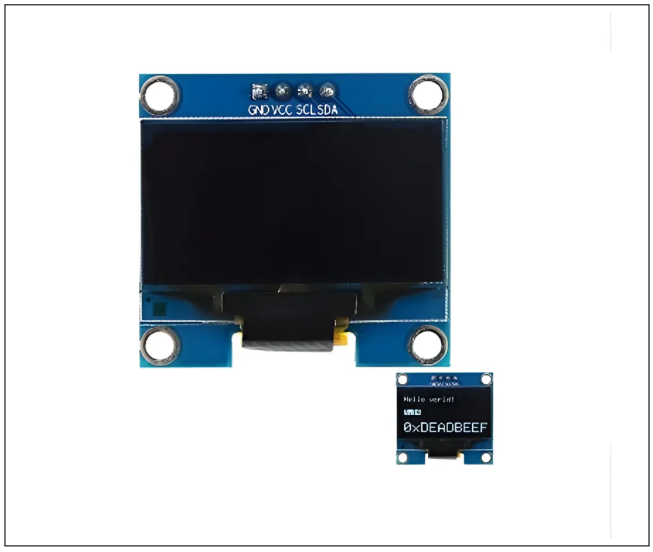


Figura 6: Display OLED SSD1306.

Encoder KY-040

Para la selección de escalas, de modos y de niveles se utilizó el encoder KY-040:

- Voltaje de operación: 5 V.
- 20 pulsos por revolución.
- Encoder incremental con pulsador.

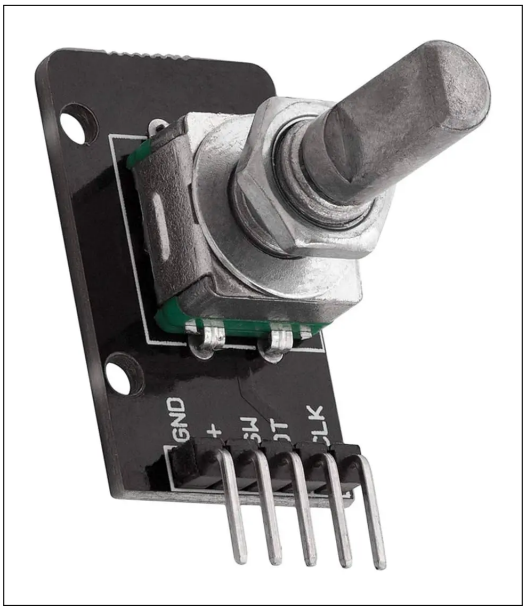


Figura 7: Encoder KY-040.

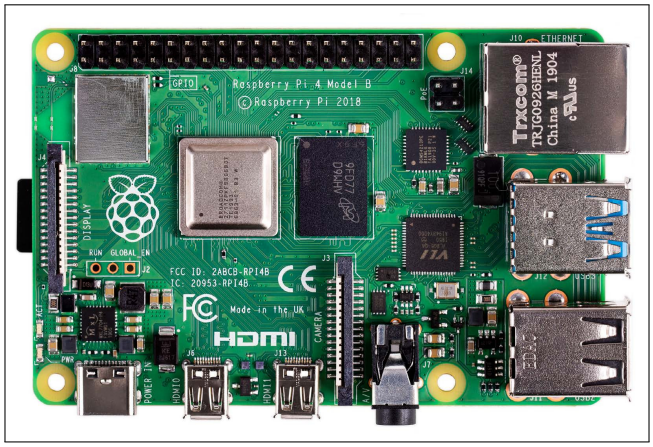


Figura 8: Dispositivo Raspberry Pi 4.

Raspberry Pi 4

- SoC (System on a Chip): Broadcom BCM2711.
- Procesador: Quad-core ARM Cortex-A72 (ARMv8) de 64 bits a 1.5 GHz.
- Memoria RAM: LPDDR4-3200 SDRAM
- Conectividad: Red Gigabit Ethernet, Wi-Fi 802.11ac de doble banda y Bluetooth 5.0.
- Puertos de E/S: Cabezal GPIO de 40 pines estándar, con multiplexor de hardware avanzado que soporta múltiples buses de comunicación (hasta 6 UARTs, 6 I2C, 5 SPI).
- Sistema Operativo: Soporte completo para distribuciones basadas en Linux (ej. Raspberry Pi OS de 64 bits), habilitando el desarrollo de módulos de kernel nativos.

Esquemáticos

Se realizó el circuito en KiCad con los jumpers para cambio de escala, las etapas de atenuación y los circuitos de protección.

2.8. Software

Lista de tareas

El flujo de datos se diseñó para separar las etapas de adquisición, procesamiento y comunicación:

- **Gestión de datos:** la tarea `vAdcTask` envía punteros de memoria a `vTriggerTask` a través de la cola `full_queue` para maximizar la velocidad de transferencia.
- **Parámetros de configuración:** se implementó un patrón de diseño tipo *Mailbox* mediante la cola `config_queue`, permitiendo que la tarea del menú actualice los parámetros

(flanco, nivel de disparo, escalas) de forma segura y no bloqueante para el motor de procesamiento. Además, cada modificación se almacena en la memoria Flash del ESP32, logrando que la configuración persista entre reinicios del sistema.

- **Comunicación asíncrona:** la tarea `vUartTask` actúa como un portero de la comunicación, recibiendo tramas completas desde `vTriggerTask` a través de `uart_queue`, evitando colisiones de datos y garantizando la fluidez de la transmisión hacia el equipo externo.

Para cumplir con los requerimientos de captura de señales de alta velocidad, se ha configurado una frecuencia de muestreo de 20 kHz. Esta tasa asegura una fidelidad óptima para la representación de las señales, permitiendo realizar el análisis de disparo y filtrado de ruido de manera eficiente antes de la visualización.

Cuadro 1: Descripción detallada de tareas

Tarea	Descripción
`vAdcTask`	Adquiere muestras crudas del driver ADC continuo (DMA) esperando *Task Notifications*, procesa los frames y encola los datos en `adc_raw_queue`.
`vTriggerTask`	Tarea DSP que procesa datos de `adc_raw_queue` aplicando escala y diezmado, detecta el flanco de disparo y encuadra la ventana de datos para transmisión.
`vUartTask`	Gestiona la transmisión asíncrona de tramas de datos procesados hacia la interfaz externa, leyendo de `uart_data_queue`.
`vMenuTask`	Gestor central de la interfaz de usuario orientado a eventos. Procesa giros, botones, comandos y flash, actualiza la configuración global en `config_queue` y el OLED.
`vButtonTask`	Espera semáforo de la ISR del botón de usuario, implementa antirrebote (*debounce*) y procesa eventos de entrada del usuario hacia el menú central.
`vComandoUartTask`	Recibe y valida bytes por UART, verifica comandos completos mediante *checksum* y envía eventos de nueva configuración al menú central.
`vEncoder`	Lee el contador del periférico PCNT (encoder rotativo), calcula la dirección del giro (+1/−1) y envía eventos de navegación/ajuste hacia el menú central.
`vFlash`	Gestiona asíncronamente la persistencia de configuraciones en memoria no volátil (NVS, blob), realizando cargas y guardados a pedido del menú central.

Interfaz Gráfica

Para el ploteo de las señales se optó finalmente por utilizar una interfaz desarrollada en Python y la librería pyqtgraph, dado que otras opciones como SerialPlot [1] restringían mucho la visualización. Además, Python ofrece más libertad para desarrollar una interfaz a gusto con cursores y valores pico.

La interfaz se caracteriza por:

- **Sincronización dinámica:** implementa un protocolo basado en una cabecera SYNC, permitiendo actualizar parámetros de control (flanco, escalas) de forma instantánea según el estado del firmware.
- **Visualización eficiente:** utiliza un gestor de eventos (`QTimer`) para el refresco del gráfico, asegurando una visualización fluida incluso bajo flujos constantes de datos.
- **Análisis de señales:** incorpora cursores dinámicos para la medición precisa de parámetros clave como tensión pico-pico, máximos y mínimos sobre la señal en tiempo real.
- **Gestión de errores:** incluye un sistema de detección de pérdida de enlace que notifica al usuario ante cualquier interrupción en la comunicación serial.

Interfaz de Consola

Para la gestión remota del instrumento, se desarrolló una aplicación de consola en lenguaje C que se ejecuta en el espacio de usuario de la Raspberry Pi. Esta aplicación actúa como un puente bidireccional asíncrono entre el driver del kernel (`/dev/esp32link`) y la conexión de red SSH establecida por el usuario o la interfaz gráfica.

Su arquitectura se basa en un diseño multihilo que separa los flujos de datos y de control. Un hilo secundario se dedica exclusivamente a extraer el flujo continuo de tramas binarias provenientes del ADC mediante llamadas bloqueantes `read()` sobre el dispositivo de caracteres, redirigiéndolas intactas hacia la salida estándar (`stdout`) para su ploteo remoto. Simultáneamente, el hilo principal actúa como intérprete de comandos: captura secuencias de texto introducidas por la entrada estándar (`stdin`), las procesa mediante un analizador sintáctico (parser) y las convierte en llamadas al sistema de control fuera de banda (`ioctl`). Este mecanismo permite reconfigurar los parámetros del osciloscopio en tiempo real sin interrumpir ni corromper el flujo masivo de adquisición de la señal.

El detalle exhaustivo de las instrucciones soportadas por esta interfaz, incluyendo los comandos de configuración, consulta de estado y restablecimiento del sistema, se encuentra documentado en el **Anexo D: Listado de comandos**.

3. Resultados

En la Figura 9 se muestra el equipo terminado y en la Figura 10 la interfaz gráfica desarrollada en Python para la visualización de las señales adquiridas.



Figura 9: Prototipo físico del equipo terminado.

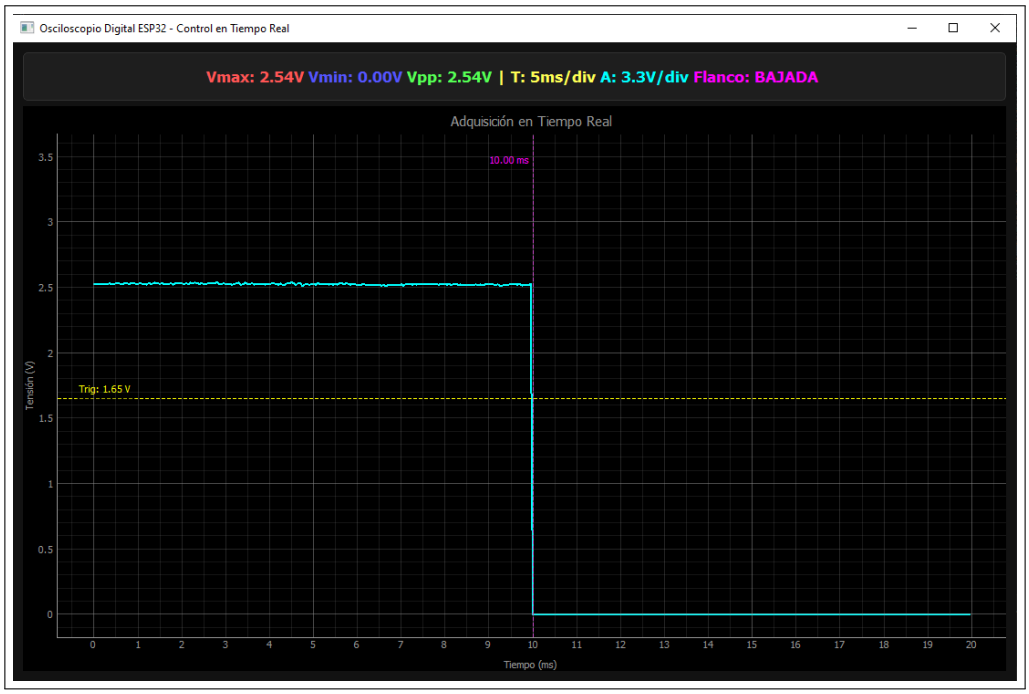


Figura 10: Interfaz gráfica del osciloscopio hecha con Python.

3.1. Mediciones

Se evaluó el instrumento frente a un osciloscopio patrón GW Instek GOS-6051. Utilizando el Modo X10 (considerado el peor caso debido a las tolerancias de la red de atenuación), se observó que el error relativo se mantiene por debajo del 2 % para señales de gran amplitud. El error máximo registrado fue del 4.58 % al medir señales de baja amplitud (2.5V), un comportamiento esperable debido al piso de ruido y la cuantificación del ADC.

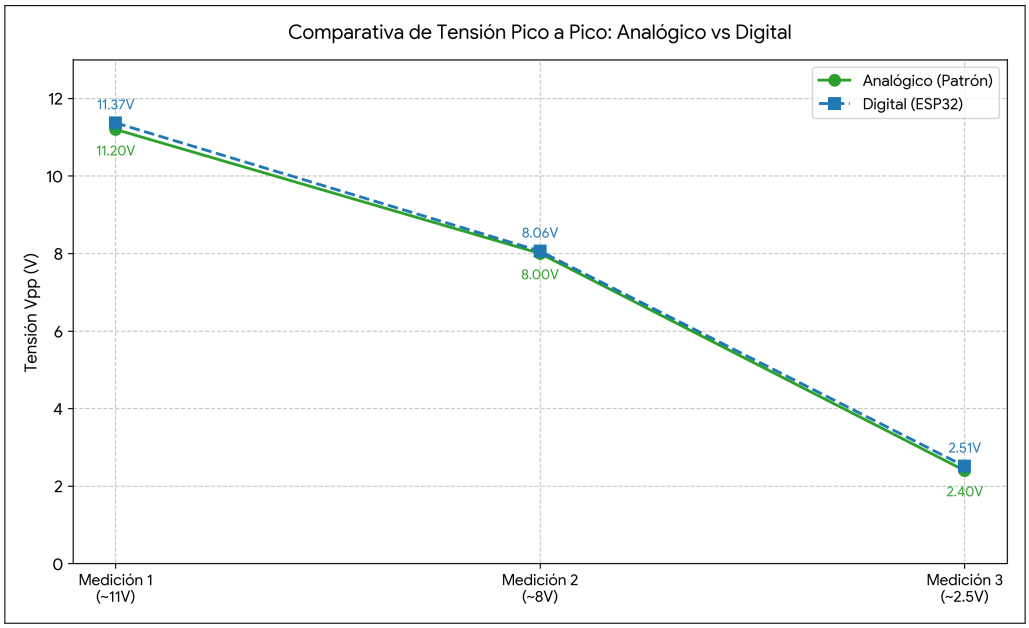


Figura 11: Comparación de mediciones en Modo x10. En verde el instrumento patrón analógico y en azul la medición hecha con el osciloscopio evaluado.

4. Conclusiones

Se lograron abordar todos los ítems planteados. Se identificaron los límites del ESP32 en cuanto al uso del ADC, lo que requirió adaptar la entrada mediante un atenuador resistivo y, en otros casos, emplear diferentes métodos de acondicionamiento de señal. Además, se aprovecharon las ventajas de contar con un sistema operativo en tiempo real como FreeRTOS, que permite sincronizar las tareas para garantizar el correcto funcionamiento del dispositivo, a diferencia de las funciones convencionales.

Asimismo, se utilizaron diversas características del ESP32-S3: el PCNT para la lectura del encoder, el ADC continuo de 12 bits para la adquisición de la señal de entrada y el UART para establecer una comunicación confiable con la aplicación.

En la segunda etapa del proyecto, se incorporó una Raspberry Pi con el objetivo de dotar al sistema de capacidad de operación remota. Esta modificación requirió adaptar distintos componentes para lograr una arquitectura más coherente y funcional. En primer lugar, los jumpers utilizados para seleccionar los modos de funcionamiento fueron reemplazados por relés, lo que permitió realizar el cambio de modo de forma remota, sin necesidad de acceder físicamente al circuito. Asimismo, se trasladó a la Raspberry Pi la responsabilidad del envío de tramas, eliminando la necesidad de mantener el ESP conectado directamente a la PC.

Como posibles mejoras futuras, se podría implementar un método de interpolación que permita aumentar la frecuencia máxima de las señales que pueden medirse. También, sería conveniente revisar el funcionamiento del modo AC, cuyo desempeño no resultó satisfactorio; sin embargo, se decidió no profundizar en este aspecto, dado que no formaba parte de los objetivos establecidos para la EGA. Por otra parte, la utilización de resistencias de mayor precisión permitiría reducir el error asociado al acondicionamiento de la señal.

A pesar de estas limitaciones, se logró desarrollar un osciloscopio digital monocanal funcional, con selección de modo de operación, configuración de trigger y escalas de medición adecuadas para la visualización y el análisis de señales de audio. De esta manera, se cumplieron los objetivos principales del proyecto y se obtuvo una plataforma funcional que permite identificar posibles mejoras y ampliaciones futuras.

5. Referencias

[1] H. Yavuz, *SerialPlot - Realtime Plotting Software*, 2023. [En línea]. Disponible en: https://hackaday.io/project/5334-serialplot-realtime-plotting-software

[2] Espressif Systems, “ESP32-S3 Wi-Fi & BLE 5 SoC”. [En línea]. Disponible en: https://www.espressif.com/en/products/socs/esp32-s3

Anexo A: Diagramas del sistema

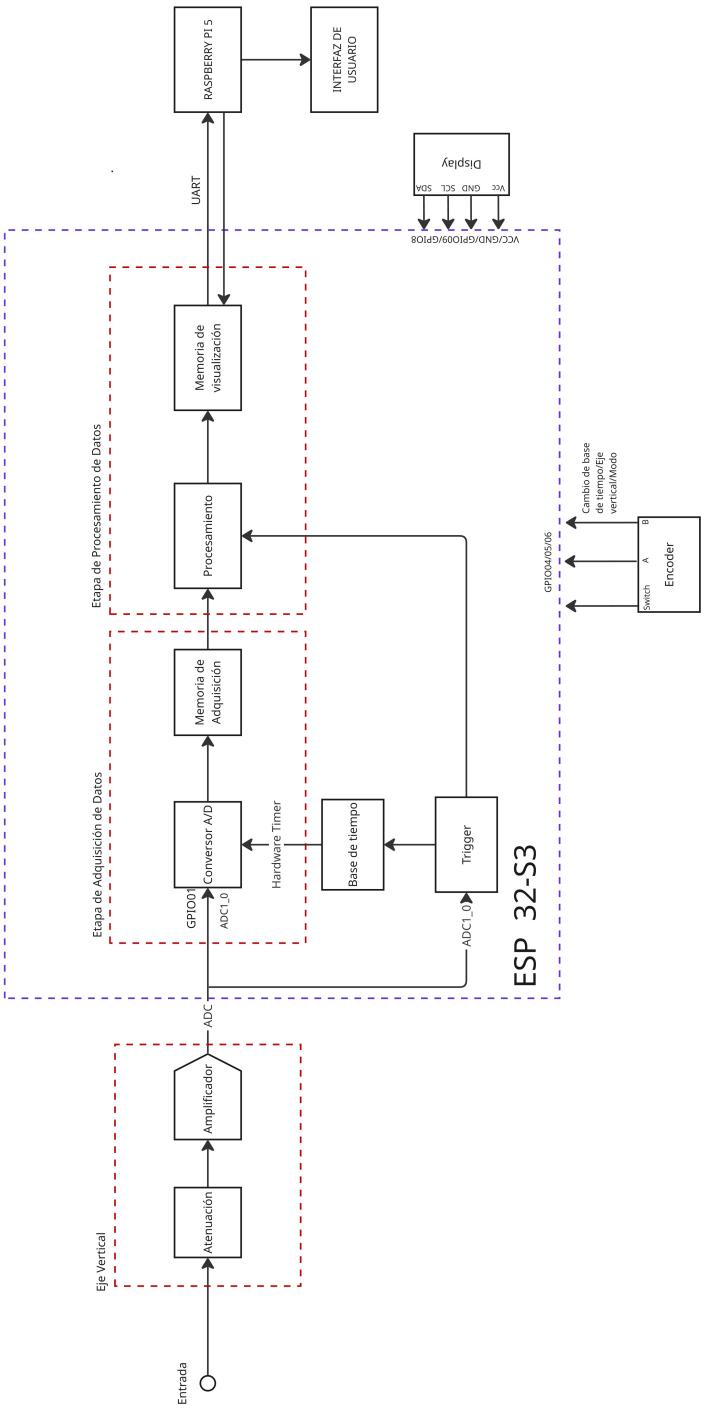


Figura 12: Diagrama en bloques.

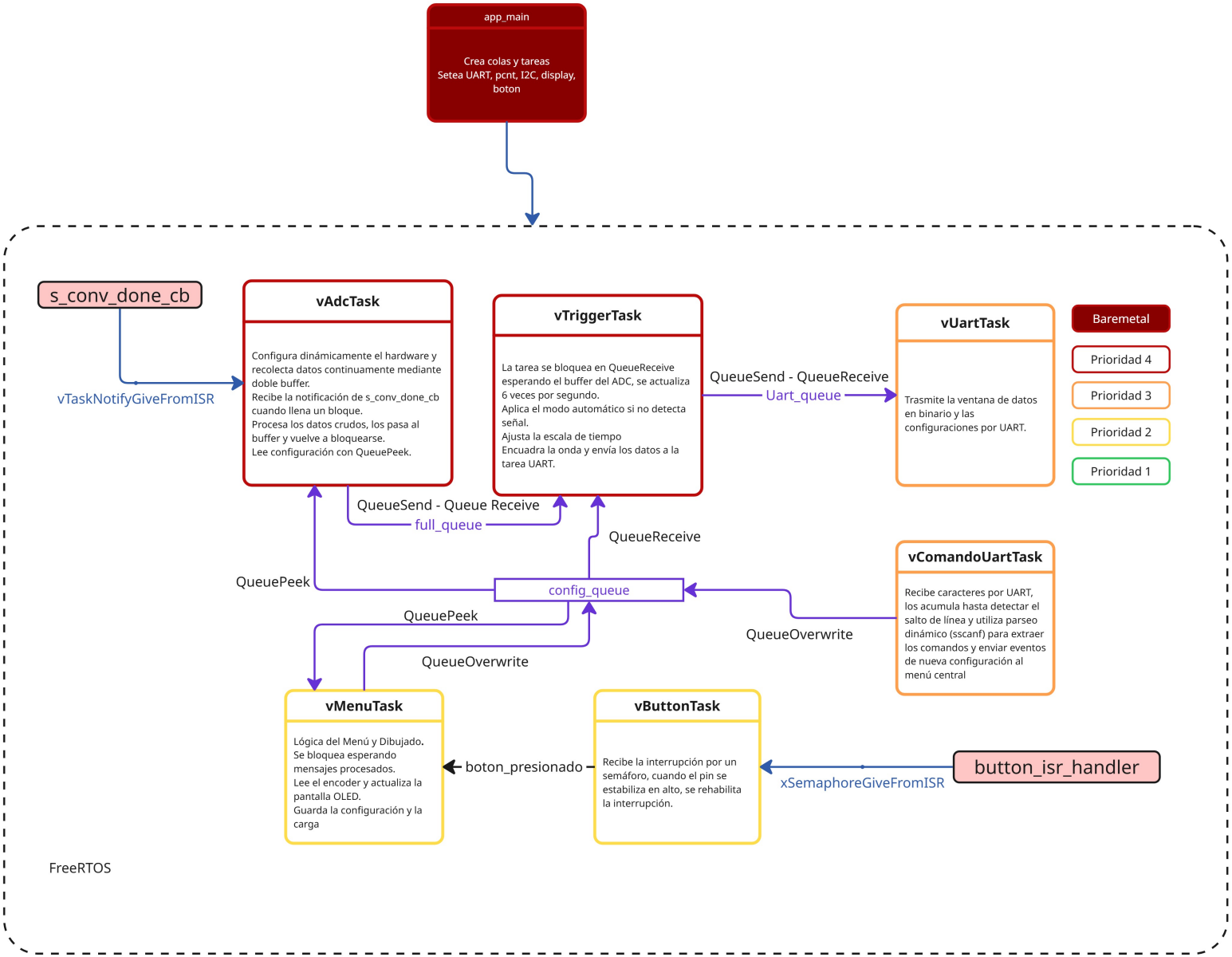


Figura 13: Diagrama de tareas.

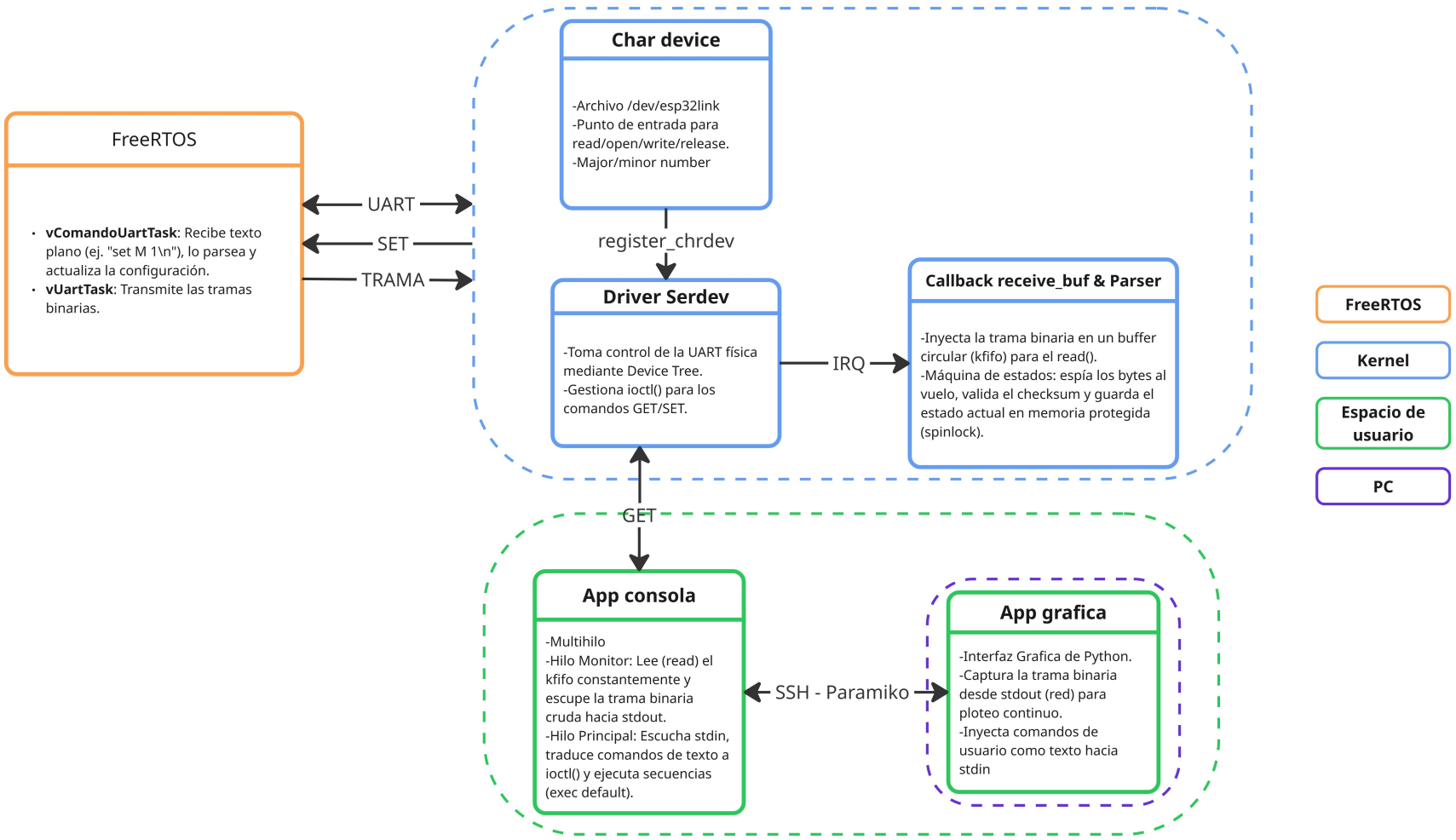


Figura 14: Diagrama de software.

Anexo B: Esquemático y PCB

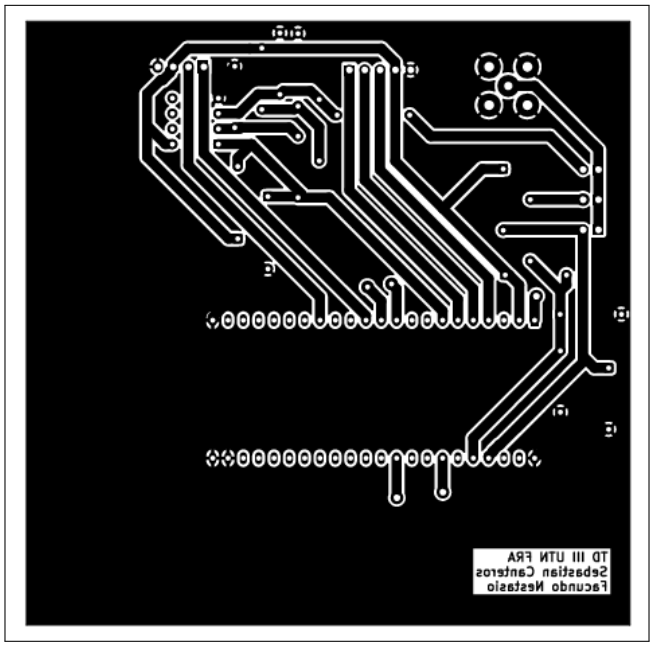


Figura 15: PCB.

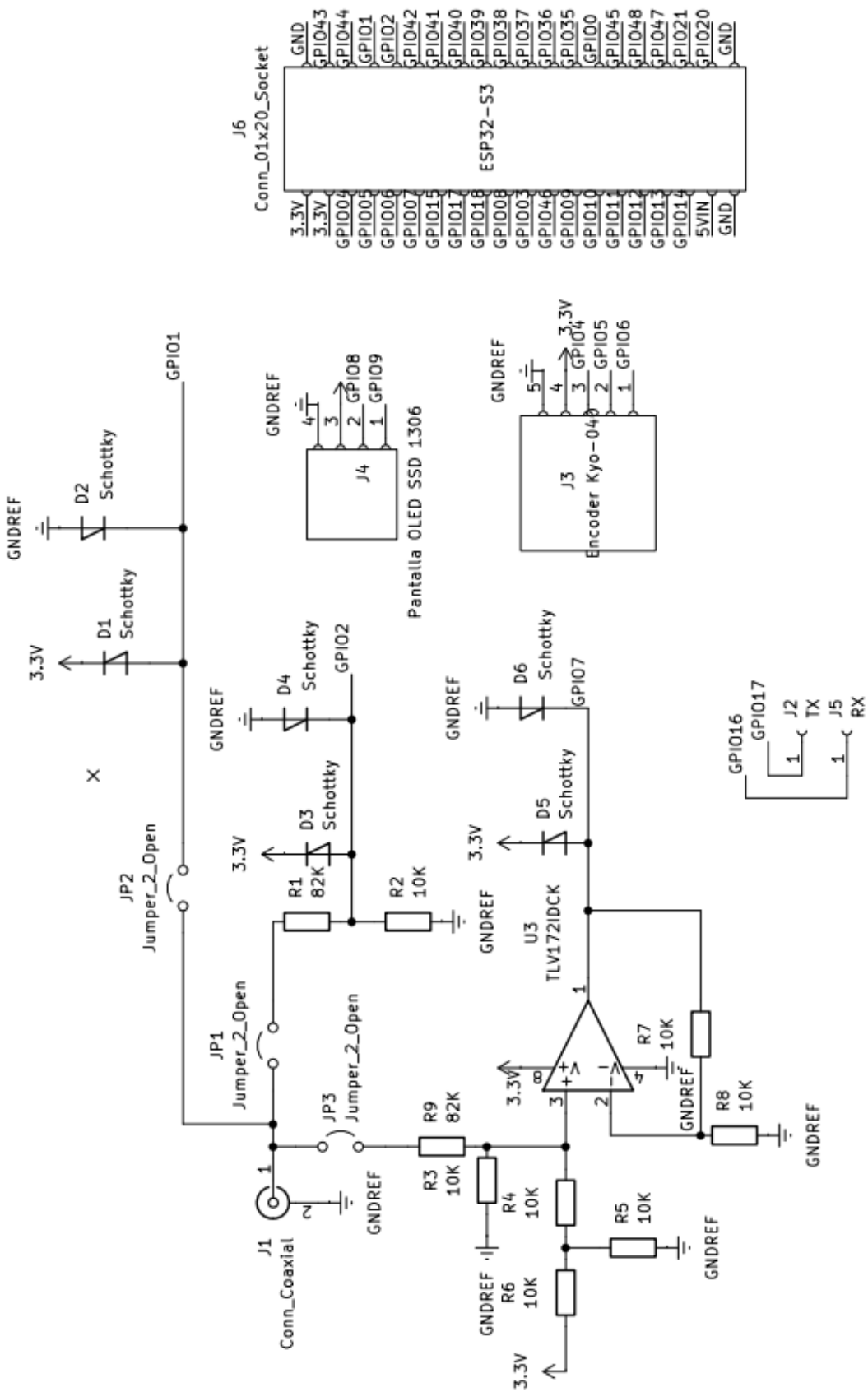


Figura 16: Esquemático.

Anexo C: Manual de usuario y dispositivo

Especificaciones

- **Tensión de alimentación:** 5V (vía conexión USB).
- **Canales analógicos:** 1 canal.
- **Modos de acoplamiento/entrada:** X1, X10, AC.
- **Escalas de amplitud (vertical):** 1.0 V/div y 5.0 V/div.
- **Bases de tiempo (horizontal):** 0.5, 1.0, 2.0 y 5.0 ms/div.
- **Sincronismo (Trigger):** Disparo por nivel configurable (0 - 4095) y selección de flanco (Subida / Bajada).
- **Visualización remota:** Transmisión de datos por UART hacia Raspberry Pi y visualización en PC mediante red (SSH).

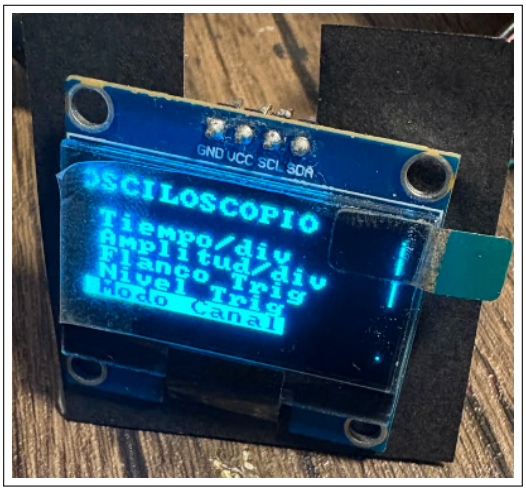


Figura 17: Menú del osciloscopio en la pantalla OLED.

- **Selector rotativo:** se puede girar (navegar y cambiar valores) y presionar (entrar a una opción y confirmar).
- **Pantalla OLED:** muestra el menú y el valor de cada parámetro.
- **Indicador de selección:** muestra la opción sobre la que está posicionado el usuario.
- **Tiempo/div:** escala horizontal (0.5, 1, 2 o 5 ms/div).
- **Amplitud/div:** escala vertical (1 o 5 V/div).
- **Flanco:** flanco de disparo del trigger (subida o bajada).
- **Nivel Trig:** nivel de tensión del trigger.

- **Modo Canal:** selección del canal de entrada.

Modo de uso general

1. Girar el selector rotativo hasta que el indicador de selección esté sobre la opción deseada.
2. Presionar el selector rotativo (el recuadro cambia de color: la opción queda en edición).
3. Girar el selector rotativo hasta el valor deseado.
4. Presionar nuevamente el selector para confirmar el cambio.

Configuración: Amplitud/div

1. Girar el selector hasta posicionar el indicador sobre **Amplitud/div**.
2. Presionar el selector.
3. Girar para alternar entre **1 V/div** y **5 V/div**.
4. Presionar para confirmar.

Configuración: Tiempo/div

1. Girar el selector hasta posicionar el indicador sobre **Tiempo/div**.
2. Presionar el selector.
3. Girar para elegir entre **0.5 ms/div**, **1 ms/div**, **2 ms/div** o **5 ms/div**.
4. Presionar para confirmar.

Configuración: Nivel de trigger

1. Girar el selector hasta posicionar el indicador sobre **Nivel Trig**.
2. Presionar el selector.
3. Girar hasta que el nivel quede dentro de la señal, de modo de mantener el sincronismo (la línea de trigger se ve en el software).
4. Presionar para confirmar.

Configuración: Flanco

1. Girar el selector hasta posicionar el indicador sobre **Flanco**.
2. Presionar el selector.
3. Girar para elegir si el trigger dispara en la **subida** o en la **bajada**.
4. Presionar para confirmar.

Configuración: Modo Canal

1. Girar el selector hasta posicionar el indicador sobre **Modo Canal**.
2. Presionar el selector.

3. Girar hasta el canal deseado.
4. Presionar para confirmar.

Interfaz de usuario

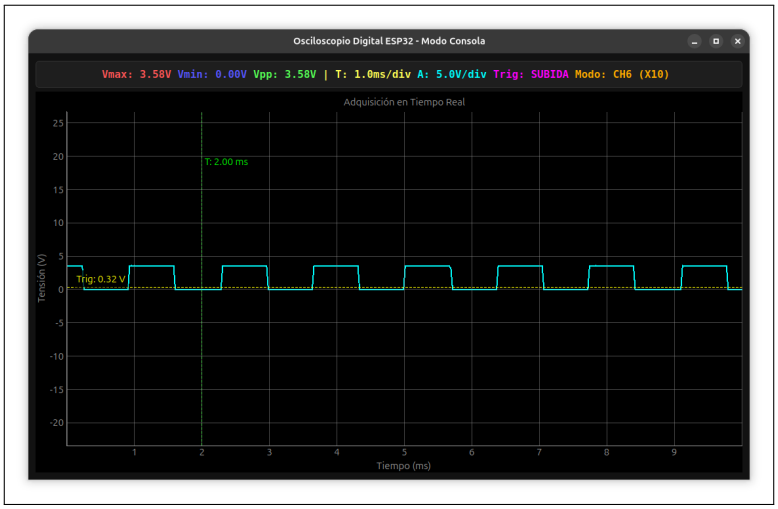


Figura 18: Interfaz de usuario.

- **Vmax / Vmin / Vpp:** tensión máxima, mínima y pico a pico de la señal.
- **T / A:** escalas actuales de tiempo/div y amplitud/div.
- **Trig:** flanco de disparo seleccionado (SUBIDA o BAJADA) y su nivel.
- **Modo:** canal de adquisición en uso.

Interfaz remota (PC y Consola)

Para el análisis y la visualización de la señal en tiempo real, el equipo cuenta con una arquitectura de red que permite operarlo desde una computadora de forma remota.

Visualización Gráfica (Aplicación Python)

- **Vista gráfica en tiempo real:** Muestra la onda adquirida con su respectiva grilla sincronizada según las escalas elegidas.
- **Cursores de medición:** Líneas dinámicas (amarilla para el Nivel de Trigger, verde para el Tiempo) que el usuario puede visualizar sobre la onda.
- **Panel de estadísticas:** Muestra en tiempo real los valores de Vmax, Vmin, Vpp (Tensión pico a pico), además de reflejar la configuración actual del hardware (T, A, Flanco, Modo).

Consola de Control por Red (SSH)

Para usuarios avanzados, el sistema operativo de la Raspberry Pi expone una interfaz de línea de comandos (CLI) que permite gobernar el osciloscopio enviando comandos de texto.

Modo de uso: Envío de comandos por consola

1. Asegúrese de estar conectado por SSH a la Raspberry Pi.
2. Ejecute la aplicación en modo interactivo ingresando `sudo ./app_pi -i`.
3. Escriba el comando deseado y presione ENTER.
    - **Ejemplo de ajuste:** Escriba `set M 2` para cambiar el modo a AC. El sistema responderá confirmando el envío.
    - **Ejemplo de lectura:** Escriba `get M` para consultar el modo actual. El sistema responderá con el valor físico (ej. `M = 2 ->Modo: AC (CH2)`).
    - **Ejemplo de diagnóstico:** Escriba `exec state` para obtener un volcado completo en pantalla del estado actual de todos los parámetros del osciloscopio.
4. Para restablecer el equipo a valores de fábrica en caso de pérdida de sincronismo, ingrese `exec default`.
5. Para salir, ingrese `salir`.

Anexo D: Listado de comandos

Comando	Descripción
`set M <valor>`	Establece el modo de entrada del instrumento (0: X1, 1: X10, 2: AC)
`set F <valor>`	Establece el flanco de disparo del trigger (0: Subida, 1: Bajada)
`set T <valor>`	Establece la escala de la base de tiempo (0: 0.5, 1: 1.0, 2: 2.0, 3: 5.0 ms/div)
`set A <valor>`	Establece la escala de amplitud vertical (0: 1.0 V/div, 1: 5.0 V/div)
`set L <valor>`	Establece el nivel de disparo del trigger en valores crudos del ADC (0 a 4095)
`get <M|F|T|A|L>`	Obtiene el valor de una variable y lo muestra traducido a su magnitud física real
`exec state`	Consulta al kernel por todas las variables e imprime un resumen de la configuración actual
`exec default`	Aplica una ráfaga de configuraciones para restablecer el equipo a sus valores predeterminados
`salir`	Cierra la aplicación de consola y termina la ejecución del programa

Anexo E: Muestra de Mediciones

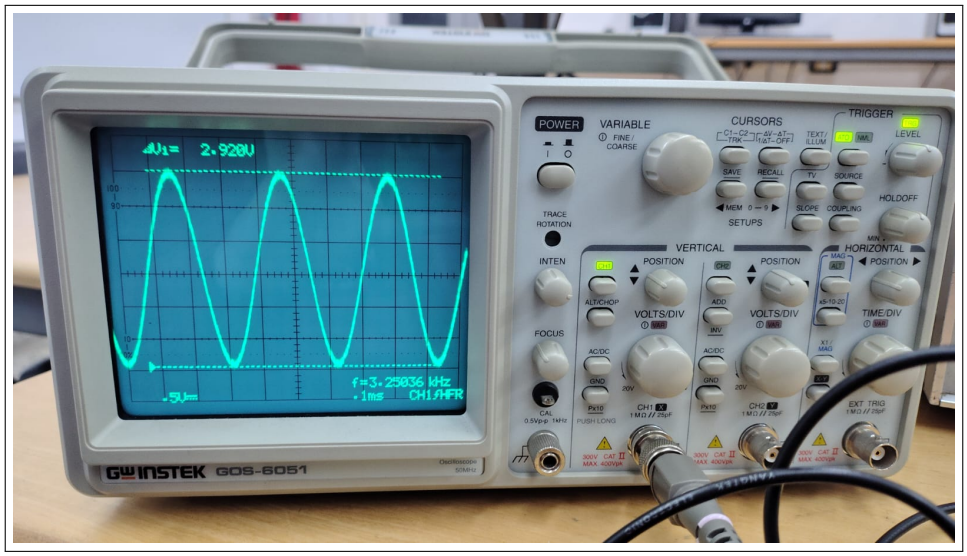


Figura 19: Medicion Modo X1 GW Instek GOS-6051

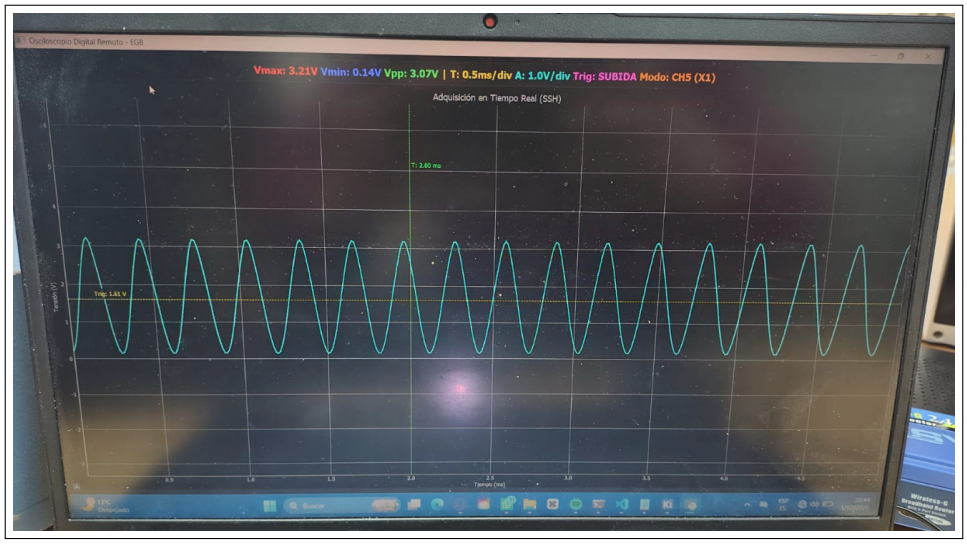


Figura 20: Medicion Modo X1 ESP32-S3

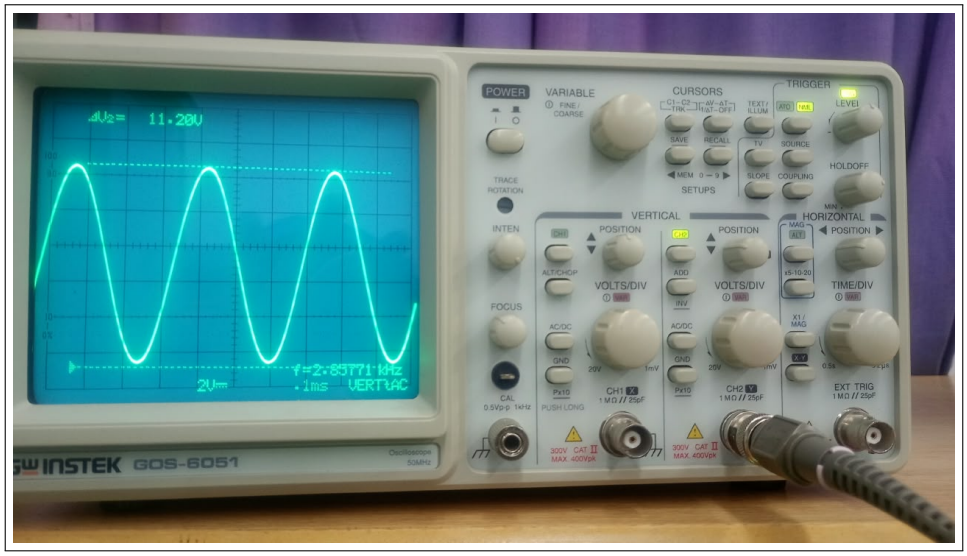


Figura 21: Medicion Modo X10 GW Instek GOS-6051



Figura 22: Modo X10 ESP32-S3